

ESTUDIO DE CASO

SISTEMA DE PUNTO DE VENTA PARA SHINGU COFFEE SHOP

Proyecto de fundamentos de programación

Adecuación del Código con veinte productos, cálculo de IVA por producto, inventario, ventas y control de errores.

lenguajes solicitados: java, c# y python

Alumno:	OLVERA HERRERA SEBASTIAN
Docente:	ALVAREZ NAVARRO EDUARDO
Materia:	FUNDAMENTOS DE PROGRAMACIÓN
Proyecto:	PUNTO DE VENTA - CAFETERÍA
Negocio:	SHINGU COFFEE SHOP
Fecha:	20/05/2026

“Códigos en GitHub en diferentes lenguajes de programación.”

Java

https://github.com/sebastianolveraherrera2004-beep/Proyecto_Integrador/blob/main/Shingu%20Coffee_codigo.java

C#

https://github.com/sebastianolveraherrera2004-beep/Proyecto_Integrador/blob/main/Shingu%20Coffee_codigoC%23

Python

https://github.com/sebastianolveraherrera2004-beep/Proyecto_Integrador/blob/main/Shingu%20Coffee_codigoPython

1.Introducción

El presente estudio de caso adapta el proyecto de punto de venta desarrollado originalmente para una tienda de abarrotes hacia una cafetería llamada Shingu Coffee Shop. El sistema conserva la lógica original del proyecto: uso de menús en consola, matrices estáticas, gestión de productos, control de inventario, registro de ventas y generación de ticket.

La modificación principal consiste en cambiar el giro del negocio, ampliar el catálogo a 20 productos de cafetería y agregar un dato nuevo por producto: el porcentaje de IVA aplicable. Además, se integran funciones para validar entradas, descontar stock, cancelar ventas y calcular el IVA según cada producto.

2. Descripción del negocio

Shingu Coffee Shop es una tienda dedicada a la venta de café, bebidas calientes, bebidas frías y alimentos complementarios como panadería y postres. El sistema POS permite al cajero administrar productos, vender artículos, controlar existencias y revisar ventas realizadas durante la operación.

3. Catálogo de 20 productos

La matriz productos se redefine para usar 20 filas y 5 columnas: código, nombre, precio, cantidad e IVA. La quinta columna almacena el porcentaje de IVA que aplica al producto.

Código	Producto	Precio	Stock	IVA %
001	Espresso	45	10	16
002	Americano	50	10	16
003	Cappuccino	65	10	16
004	Latte	68	10	16
005	Mocha	72	10	16
006	Flat White	70	10	16
007	Caramel Macchiato	78	10	16
008	Cold Brew	75	10	16
009	Frappe de Cafe	82	10	16
010	Affogato	80	10	16
011	Chocolate Caliente	60	10	16
012	Te Chai Latte	70	10	16
013	Matcha Latte	76	10	16
014	Tisana Frutal	58	10	16
015	Croissant Mantequilla	42	10	0
016	Panini Jamon y Queso	85	10	16
017	Bagel con Queso Crema	55	10	0
018	Cheesecake Rebanada	68	10	16
019	Brownie	48	10	0
020	Galletas Artesanales	40	10	0

4. Modificaciones del proyecto

Actividad	Adecuación realizada
Actividad 1	Crear una función de lectura validada que solicite datos, limpie la entrada y repita la captura hasta recibir un dato correcto.
Actividad 2	Cargar 20 productos e incorporar la columna de porcentaje de IVA en la matriz de productos.
Actividad 3	Crear la función DescontarStock para actualizar existencias y devolver códigos de resultado según el error o éxito.
Actividad 4	Crear el procedimiento CancelarVenta para devolver productos al inventario y limpiar el ticket.
Actividad 5	Modificar el cálculo de IVA para que dependa del porcentaje de IVA de cada producto.

5. Algoritmos y funciones creadas

5.1 Función LeerValidado

La función LeerValidado solicita un dato al usuario mediante Dialogo, limpia la entrada y valida el tipo de dato con EvaluarNumerico. Si el dato es incorrecto, muestra un mensaje de error y repite la solicitud hasta obtener un valor válido.

- Recibe como parámetros texto_dialogo y tipo_dato.
- Llama internamente a Dialogo y EvaluarNumerico.
- Regresa el dato limpio y validado.

Código Java: LeerValidado

```
public static String LeerValidado(String texto_dialogo, int tipo_dato) throws IOException {
    String dato;
    do {
        dato = Dialogo(texto_dialogo);
        if (dato != null) {
            dato = dato.trim();
        }
        if (dato == null || dato.isEmpty()) {
            System.out.println("Error: dato vacio");
        } else if (!EvaluarNumerico(dato, tipo_dato)) {
            System.out.println("Error: tipo de dato incorrecto");
        } else {
            return dato;
        }
    } while (true);
}
```

5.2 Función CargarProductos con 20 productos e IVA

La función CargarProductos ahora inicializa la matriz con 20 productos de cafetería y agrega una quinta columna para el porcentaje de IVA.

Código Java: CargarProductos

```
public static String[][] CargarProductos() {
    String[][] producto = {
        { "001", "Espresso", "45", "10", "16" },
        { "002", "Americano", "50", "10", "16" },
        { "003", "Cappuccino", "65", "10", "16" },
        { "004", "Latte", "68", "10", "16" },
        { "005", "Mocha", "72", "10", "16" },
        { "006", "Flat White", "70", "10", "16" },
        { "007", "Caramel Macchiato", "78", "10", "16" },
        { "008", "Cold Brew", "75", "10", "16" },
        { "009", "Frappe de Cafe", "82", "10", "16" },
        { "010", "Affogato", "80", "10", "16" },
        { "011", "Chocolate Caliente", "60", "10", "16" },
        { "012", "Te Chai Latte", "70", "10", "16" },
        { "013", "Matcha Latte", "76", "10", "16" },
        { "014", "Tisana Frutal", "58", "10", "16" },
        { "015", "Croissant Mantequilla", "42", "10", "0" },
        { "016", "Panini Jamon y Queso", "85", "10", "16" },
        { "017", "Bagel con Queso Crema", "55", "10", "0" },
        { "018", "Cheesecake Rebanada", "68", "10", "16" },
        { "019", "Brownie", "48", "10", "0" },
        { "020", "Galletas Artesanales", "40", "10", "0" }
    };
    return producto;
}
```

5.3 Función MostrarProducto actualizada

La función MostrarProducto se actualiza para mostrar la quinta columna correspondiente al porcentaje de IVA.

Código Java: MostrarProducto

```
public static String MostrarProducto(String[] vproducto) {
    String codigo = RellenarEspacios(vproducto[0], 5);
    String producto = RellenarEspacios(vproducto[1], 30);
    String precio = RellenarEspacios(vproducto[2], 10);
    String cantidad = RellenarEspacios(vproducto[3], 10);
    String iva = RellenarEspacios(vproducto[4], 10);
    String cadena = codigo.concat(producto + precio + cantidad + iva);
    return cadena;
}
```

5.4 Función DescontarStock

La función DescontarStock centraliza la operación de disminuir el inventario. Esto evita repetir lógica dentro del menú de venta y permite detectar errores de forma clara.

- Regresa -2 si no existe el producto.
- Regresa -1 si la cantidad solicitada es mayor al stock disponible.
- Regresa 0 si el producto tiene stock igual a cero.
- Regresa 1 si el stock fue actualizado correctamente.

```
public static int DescontarStock(String[][] mproductos, String codigo, int cantidad) {
    int pos = ExisteProducto(codigo, mproductos);
    if (pos == -1) {
        return -2;
    }
    int stock = Integer.parseInt(mproductos[pos][3]);
    if (stock == 0) {
        return 0;
    }
    if (cantidad > stock) {
        return -1;
    }
    mproductos[pos][3] = String.valueOf(stock - cantidad);
    return 1;
}
```

5.5 Procedimiento CancelarVenta

CancelarVenta abstrae la lógica de cancelación. Si el ticket no se paga, devuelve los productos al inventario y limpia completamente la matriz del ticket.

Código Java: CancelarVenta

```
public static void CancelarVenta(String[][] mticket, String[][] mproductos) {
    DevolucionTicket(mticket, mproductos);
    for (int i = 0; i < mticket.length; i++) {
        for (int j = 0; j < mticket[i].length; j++) {
            mticket[i][j] = null;
        }
    }
    System.out.println("Venta cancelada. Productos devueltos al inventario.");
}
```

5.6 Función ObtenerIva e IvaTicket

Para calcular el IVA por producto, se obtiene el código del producto desde el ticket, se localiza en la matriz de productos y se recupera su porcentaje de IVA. Después se calcula el IVA total acumulando el impuesto de cada producto.

Código Java: ObtenerIva e IvaTicket

```
public static double ObtenerIva(String codigo, String[][] mproductos) {
    int pos = ExisteProducto(codigo, mproductos);
    if (pos > -1) {
        return Double.parseDouble(mproductos[pos][4]);
    }
    return 0;
}

public static double IvaTicket(String[][] mticket, String[][] mproductos) {
    double ivatotal = 0;
    int pos = ObtenerUltimaPosicion(mticket);
    for (int ciclo = 0; ciclo <= pos; ciclo++) {
        String codigo = mticket[ciclo][0];
        double iva = ObtenerIva(codigo, mproductos);
        if (iva != 0) {
            double precio = Double.parseDouble(mticket[ciclo][2]);
            double cantidad = Double.parseDouble(mticket[ciclo][3]);
            ivatotal = ivatotal + ((precio * cantidad) * (iva / 100));
        }
    }
    return ivatotal;
}
```

6. Corrida del sistema con detección de errores

En esta prueba se valida que el sistema detecte errores sin interrumpir la ejecución. Se simulan entradas incorrectas en el menú, códigos inexistentes y cantidades mayores al stock.

Ejemplo de corrida con errores

Menu de Punto de Venta - Shingu Coffee Shop

```
1.-Productos
2.-Punto de Venta
3.-Inventario
4.-Ventas
5.-Salida
Que opcion deseas : 9
No existe esta opcion
```

Menu de Punto de Venta

```
1.-Agregar
2.-Eliminar
3.-Listado
4.-Pagar
5.-Salida
Que opcion deseas : 1
Introduce el codigo del producto : 999
el codigo no existe no se puede agregar
```

```
Introduce el codigo del producto : 001
Producto agregado: Espresso
```

```
Introduce el codigo del producto : 001
Resultado DescontarStock: 1
```

```
Prueba de stock:
Cantidad solicitada: 50
Resultado DescontarStock: -1
Mensaje: La cantidad solicitada es mayor al stock disponible.
```

7. Corrida del sistema sin errores

En esta corrida se realiza una venta completa con productos válidos, cálculo de subtotal, IVA y total.

Ejemplo de corrida sin errores

```
Menu de Punto de Venta - Shingu Coffee Shop
1.-Productos
2.-Punto de Venta
3.-Inventario
4.-Ventas
5.-Salida
Que opcion deseas : 2

Fecha del Dia 19-05-2026 Ticket No 001
1.-Agregar
2.-Eliminar
3.-Listado
4.-Pagar
5.-Salida
Que opcion deseas : 1
Introduce el codigo del producto : 001
Producto agregado: Espresso

Que opcion deseas : 1
Introduce el codigo del producto : 015
Producto agregado: Croissant Mantequilla

Que opcion deseas : 4
Fecha 19-05-2026 Ticket No. 001
001 Espresso 45 1 45.00
015 Croissant Mantequilla 42 1 42.00

El total sin iva 87.00
el iva total es 7.20
el total de la venta fue 94.20
```

8. Entregables por lenguaje

Lenguaje Descripción Liga de GitHub

Java Se mantiene el código base con la lógica y nombres de variables originales. Se agregan las funciones indicadas y se actualizan las matrices

C# Se replica la lógica del sistema de consola usando arreglos bidimensionales, métodos estáticos y nombres equivalentes a los de Java.

Python Se replica la lógica usando listas bidimensionales y funciones con los mismos nombres conceptuales para productos, ventas, inventario y ticket.

9. Uso de inteligencia artificial

Para esta actividad se utilizó inteligencia artificial como apoyo para organizar el estudio de caso, la IA ChatGPT se utilizó para analizar el código buscando algún error .

10. Conclusión

La adaptación del sistema POS a Shingu Coffee Shop permite demostrar cómo una misma lógica de programación puede reutilizarse en diferentes tipos de negocios. El proyecto conserva la estructura original de menús en consola y matrices estáticas, pero amplía sus capacidades al incorporar 20 productos, IVA por producto, control de stock, cancelación de venta y detección de errores. Con esto se fortalece el uso de abstracción, reconocimiento de patrones y pensamiento computacional en la solución de problemas reales.

11. Referencias del proyecto

- Estudio de caso proporcionado por el docente: sistema de punto de venta POS con productos, inventario, ventas y ticket.
- Documento de actividades de fundamentos: validación de datos, 20 productos, IVA, DescontarStock, CancelarVenta e IvaTicket.
- Página de referencia del Negocio: Shingu Coffee Shop en Facebook.

11.1 LINK

1. [Libros/Pensamiento Computacional en Práctica La Lógica Universal del Código a C++, Java, C#, JavaScript y Python/Codigo Java/Capitulo7.java at main · ccalvar/Libros · GitHub](#)
2. <https://www.facebook.com/p/Shing%C5%AB-Coffee-Shop-61558462029869/>



actividades_fundame
ntos.docx



EstudiodeCaso.pdf